Alocação Dinâmica

Programação e Desenvolvimento de Software I
Heitor Ramos

Vetores alocados em tempo de execução

```
int n;
scanf(“%d”, &n);
int vetor[n];
int *vetor = (int *) malloc(n*sizeof(int));
….
free(vetor);
```

Esquema de um Programa na Memória

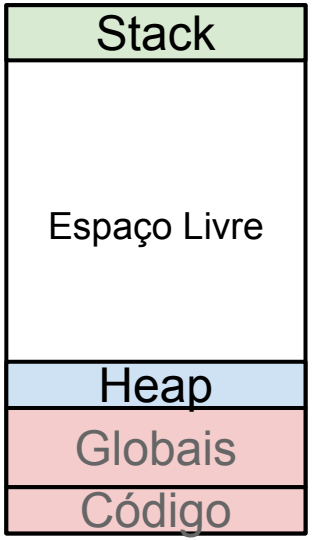
- Podemos simplificar um programa com um esquema como o abaixo

- [Sempre] Código
  - Que ocupa espaço

- [Pode ter] Variáveis globais
  - Que ocupa espaço

- [Sempre] Stack ou Pilha
  - Utilizado pelo hardware para chamar funções

- [Sempre] Heap
  - Alocação dinâmica (esta aula)



Stack/Pilha

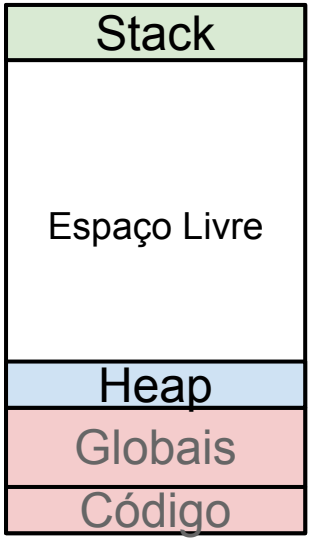
Então sabemos o que é a pilha. Onde as "funções crescem"

No esquema anterior, desenhamos a pilha crescendo para baixo

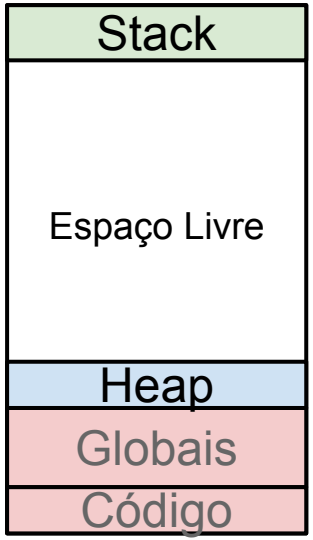
O oposto também é comum (depende do livro/arquitetura)

Heap

E esse heap?!



Heap



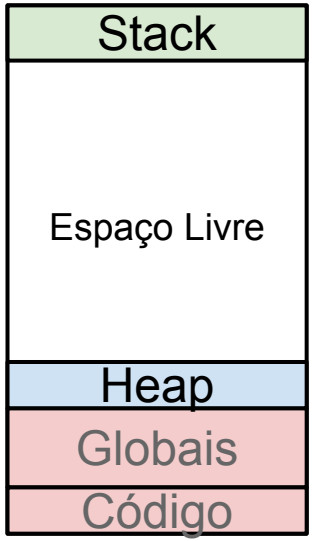
E esse heap?!

- Utilizado na alocação dinâmica
- Nem sempre sabemos o tamanho que vamos precisar
  - Por isso dinâmico

- Podemos usar o mesmo para passar vetores de um lado para o outro
  - Na verdade, alocamos memória no heap
  - Tal memória é controlada pelo usuário
  - “Raw memory”
    - vetores, structs, tipos, etc

Malloc

- Por motivos como o acima, alocação dinâmica é útil
- malloc/free



Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*)malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
	0x0044	
	0x0040	
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
	0x0040	
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	??
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

malloc aloca espaço no heap

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

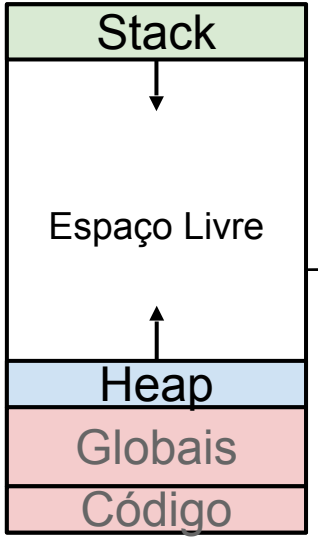
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



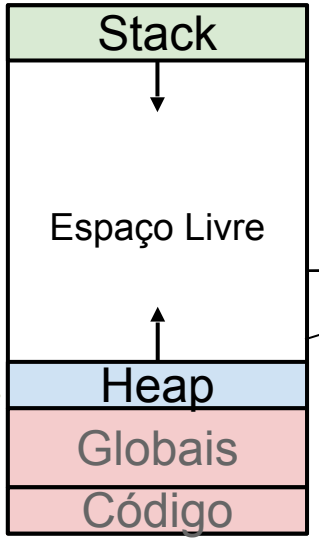
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Utilizando o heap com malloc

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



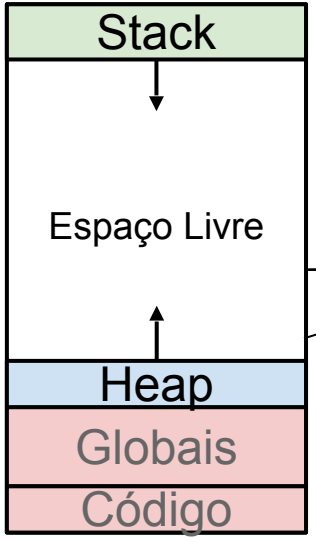
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

apontamos para o local do heap

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



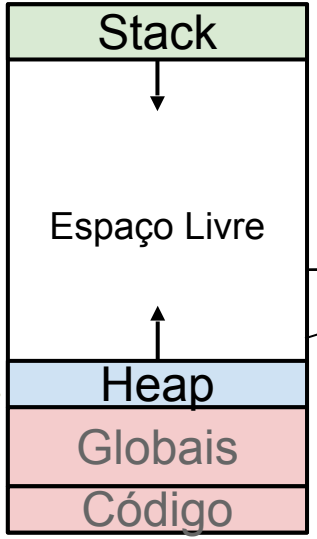
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

setamos o valor

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



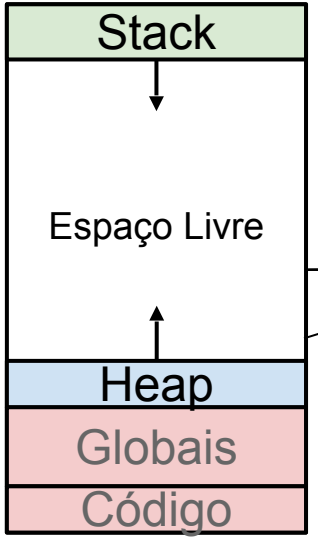
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	77
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

a função free limpa partes do heap

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	77
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

a função free limpa partes do heap

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

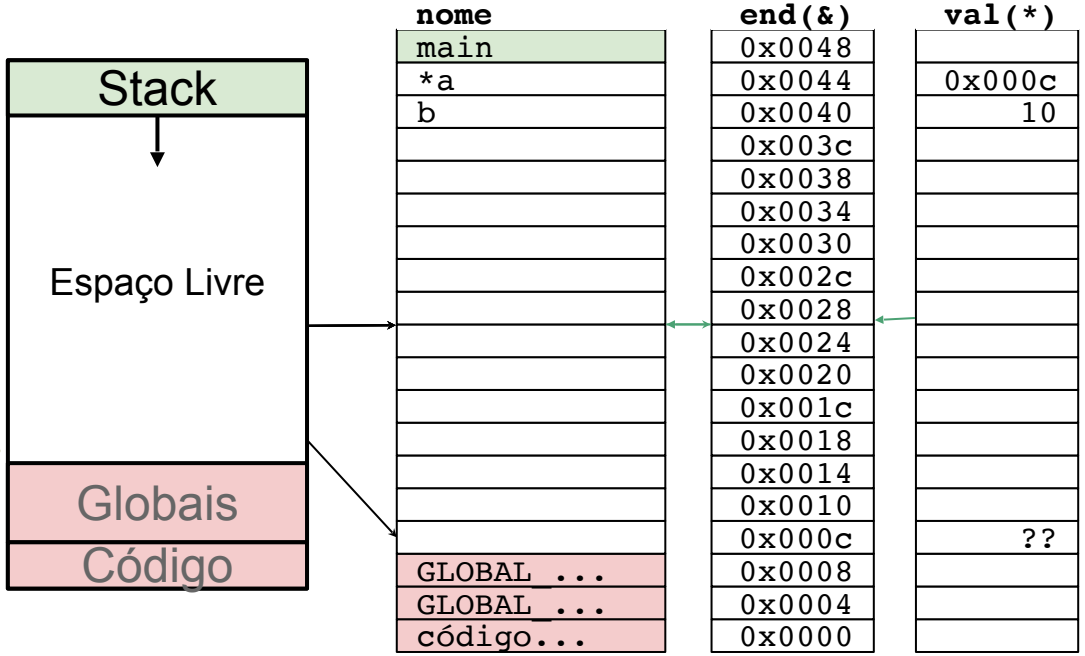
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	??
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

agora **a** aponta para lixo

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

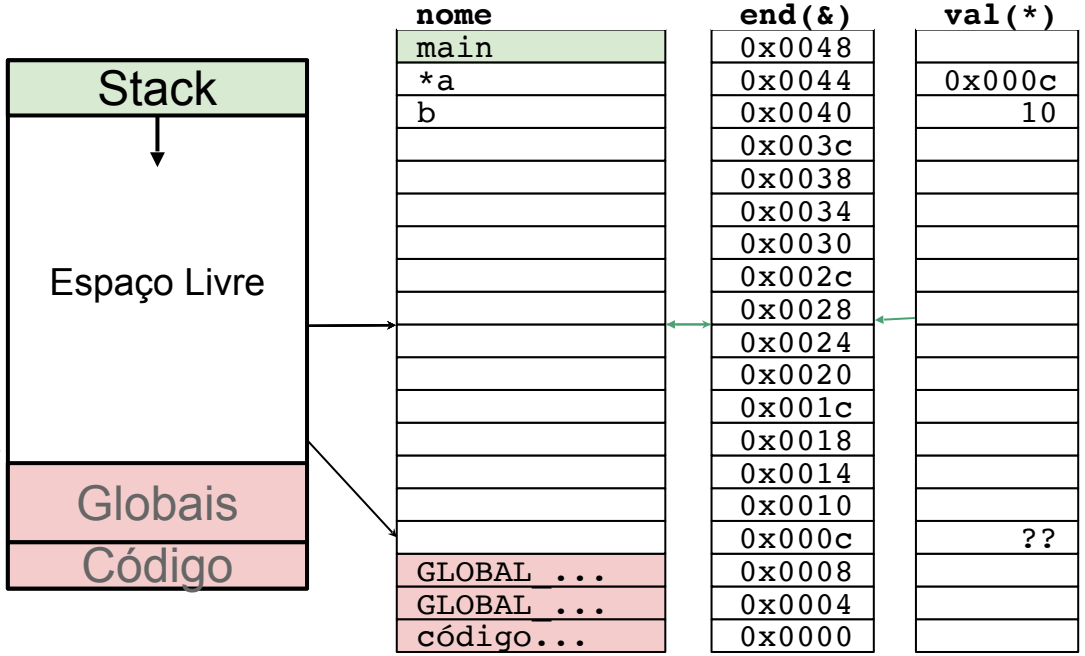


e agora?!

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```

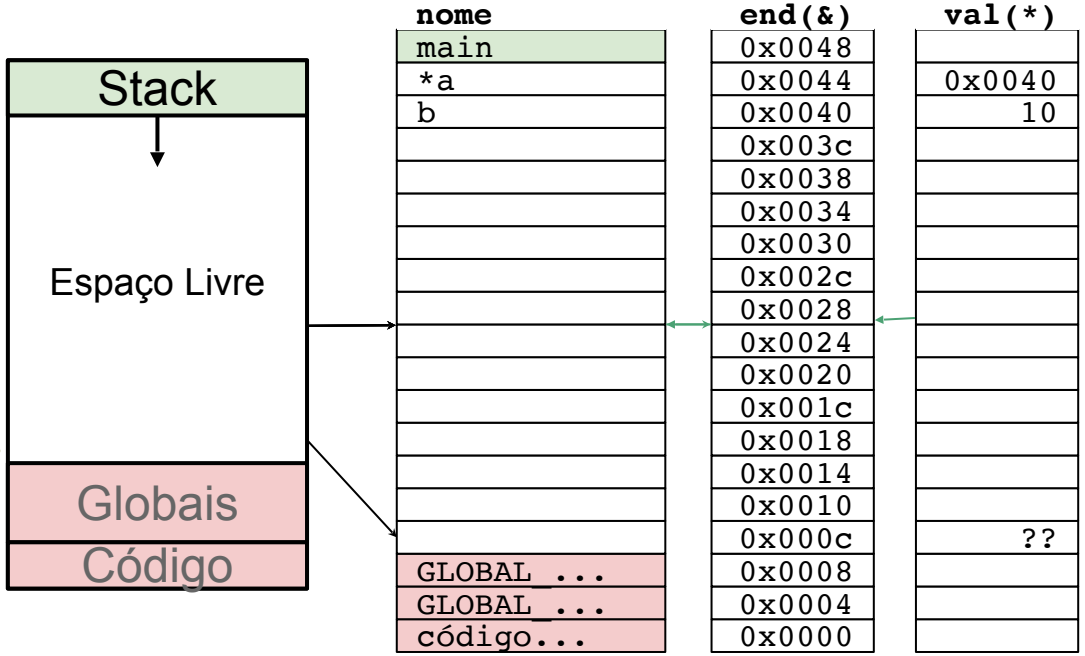


e agora?!

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(sizeof(int));
  *a = 77;
  free(a);
  a = &b;
  return 0;
}
```



alocando vetores

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(7*sizeof(int));
  a[5] = 77;
  free(a);
  a = &b;
  return 0;
}
```

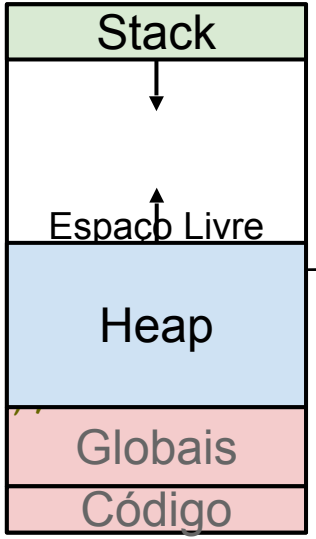
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

alocando vetores

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(7*sizeof(int));
  a[5] = 77;
  free(a);
  a = &b;
  return 0;
}
```



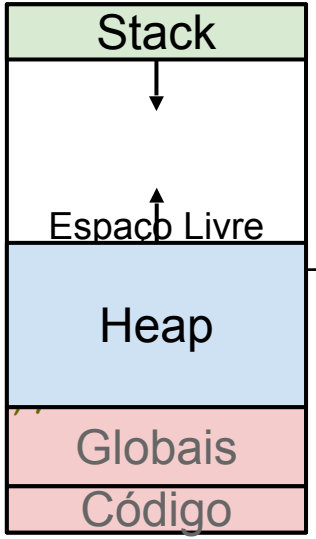
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	??
	0x0020	??
	0x001c	??
	0x0018	??
	0x0014	??
	0x0010	??
	0x000c	??
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

o acesso é similar a um vetor

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(7*sizeof(int));
  a[5] = 77;
  free(a);
  a = &b;
  return 0;
}
```



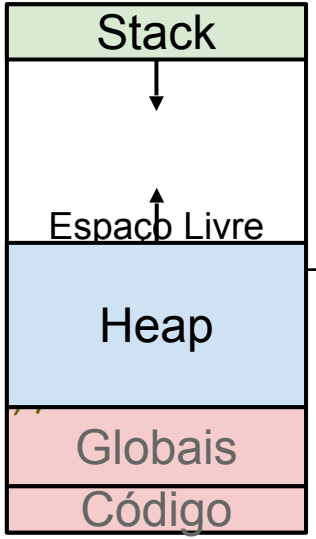
nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	??
	0x0020	77
	0x001c	??
	0x0018	??
	0x0014	??
	0x0010	??
	0x000c	??
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Liberando a memória com o free()

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(7*sizeof(int));
  a[5] = 77;
  free(a);
  a = &b;
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
*a	0x0044	0x000c
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	??
	0x0020	77
	0x001c	??
	0x0018	??
	0x0014	??
	0x0010	??
	0x000c	??
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

Memória é liberada!

```c
#include <stdio.h>
#include <stdlib.h>

// algumas globais aqui...


int main(void) {
  int *a;
  int b;
  b = 10;
  a = (int*) malloc(7*sizeof(int));
  a[5] = 77;
  free(a);
  a = &b;
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
*a	0x0044	??
b	0x0040	10
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
GLOBAL_...	0x0008	
GLOBAL_...	0x0004	
código...	0x0000	

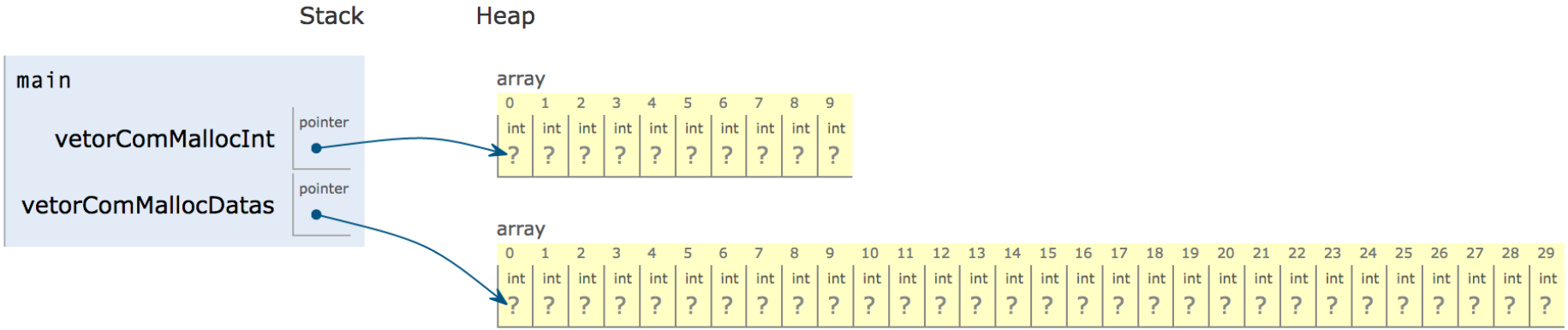
Podemos...

- Alocar vetores
- Structs
- Vetores de structs!
- Ocupam um tamanho proporcional do heap

```c
#include <stdlib.h>

typedef struct { int dia; int mes; int ano; } data_t;

int main(void) {
  int *vetorComMallocInt = (int*) malloc(10*sizeof(int));
  data_t *vetorComMallocDatas = (data_t*) malloc(10*sizeof(data_t));
  free(vetorComMallocInt);
  free(vetorComMallocDatas);
}
```



Free!

- Libere a memória do seu programa
- Free the mallocs!
- Função
  - free()
- Memory Leaks
  - Memória que não foi liberada
  - Seu programa é mais pesado do que deveria

Problema 1

Alocar um vetor de ints/datas/pontos (ou qualquer outro tipo/struct) e retornar.

Problema 1: precisamos do stdlib

```
#include <stdlib.h>

int *cria_vetor(int n) {
  int *vetor = (int *) malloc(n * sizeof(int));
  return vetor;
}
int main(void) {
  int n = 5;
  int *vetor_novo = cria_vetor(n);
  for (int i = 0; i < n; i++)
    vetor_novo[i] = i*2;
  free(vetor_novo);
}
```

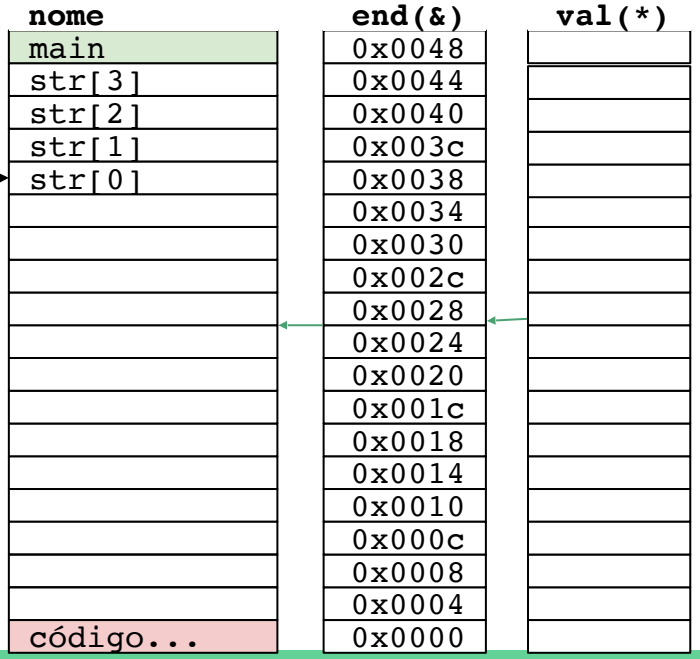
Vetores e Mallocs

Em C, tanto vetores quanto mallocs são ponteiros para a posição 0. Por isso o scanf com strings não precisa de &.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
  char str[4];
  scanf("%s", str);
}
```

str é um ponteiro para 0x0038



Aritmética de Ponteiros

```
#include <stdio.h>
#include <stdlib.h>

// qual a saída?!
int main(void) {
  int *vetor = (int*) malloc(10 * sizeof(int));
  for (int i = 0; i < 10; i++) *(vetor+i) = i*2;

  *vetor = 92; // seta a posição 0 para 92. vetor <-> &vetor[0]
  for (int i = 0; i < 10; i++) {
    printf("%d\n", *vetor);//caminha para frente. inc a memória
    vetor++;
  }
}
```

Outras Funções

calloc ➔ aloca um espaço de memória zerando o mesmo

realloc ➔ cresce/decresce um espaço já alocado

Calloc e Realloc

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int main(){
04    //alocação com malloc
05    int *p;
06    p = (int *) malloc(50*sizeof(int));
07    if(p == NULL){
08        printf("Erro: Memoria Insuficiente!\n");
09    }
10    //alocação com calloc
11    int *p1;
12    p1 = (int *) calloc(50,sizeof(int));
13    if(p1 == NULL){
14        printf("Erro: Memoria Insuficiente!\n");
15    }
16    system("pause");
17    return 0;
18  }
```

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int main(){
04    int i;
05    int *p = malloc(5*sizeof(int));
06    for (i = 0; i < 5; i++){
07        p[i] = i+1;
08    }
09    for (i = 0; i < 5; i++){
10        printf("%d\n",p[i]);
11    }
12    printf("\n");
13    //Diminui o tamanho do array
14    p = realloc(p,3*sizeof(int));
15    for (i = 0; i < 3; i++){
16        printf("%d\n",p[i]);
17    }
18    printf("\n");
19    //Aumenta o tamanho do array
20    p = realloc(p,10*sizeof(int));
21    for (i = 0; i < 10; i++){
22        printf("%d\n",p[i]);
23    }
24    system("pause");
25    return 0;
26  }
```

Dicas

Embora seja raro, um malloc pode falhar. Testamos isso verificando se o retorno é NULL. Um valor NULL indica um ponteiro para nada!

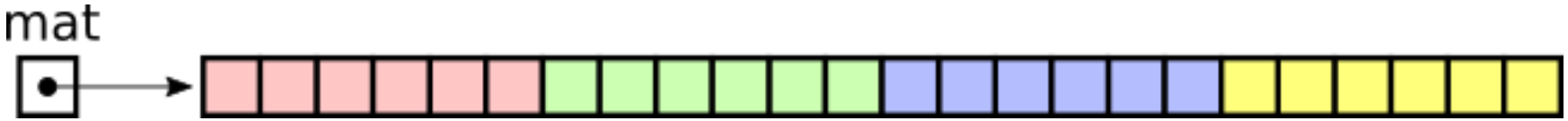
Sempre faça:

```
int *data = (int *) malloc(10 * sizeof(int));
if (data == NULL) {
  printf("Erro, sem memória!\n");
  exit(1);
}
```

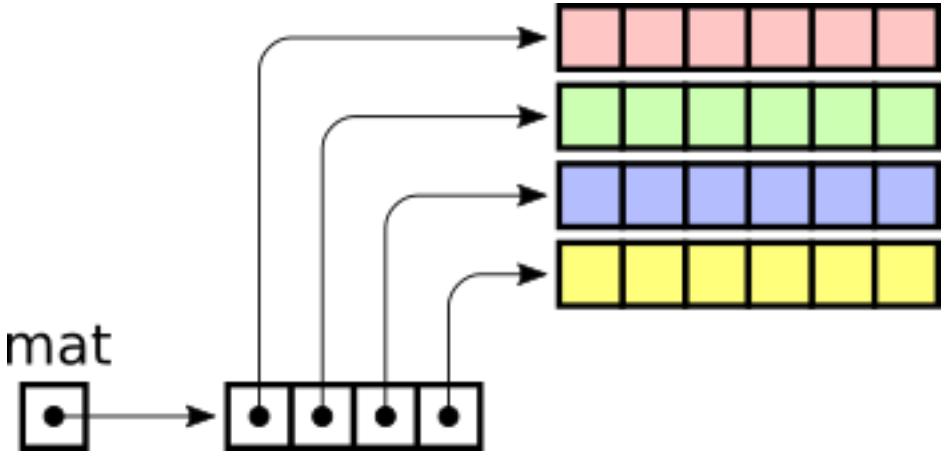
Exercício 1

Faça um programa onde o usuário indica o número de linhas e colunas de uma matriz. Após definir tais valores, escreva uma função para criar essa matriz dinamicamente (malloc). Em seguida, escreva outra função para preencher essa matriz com números lidos com scanf. Por fim, implemente outra função para desenhar a matriz na tela.

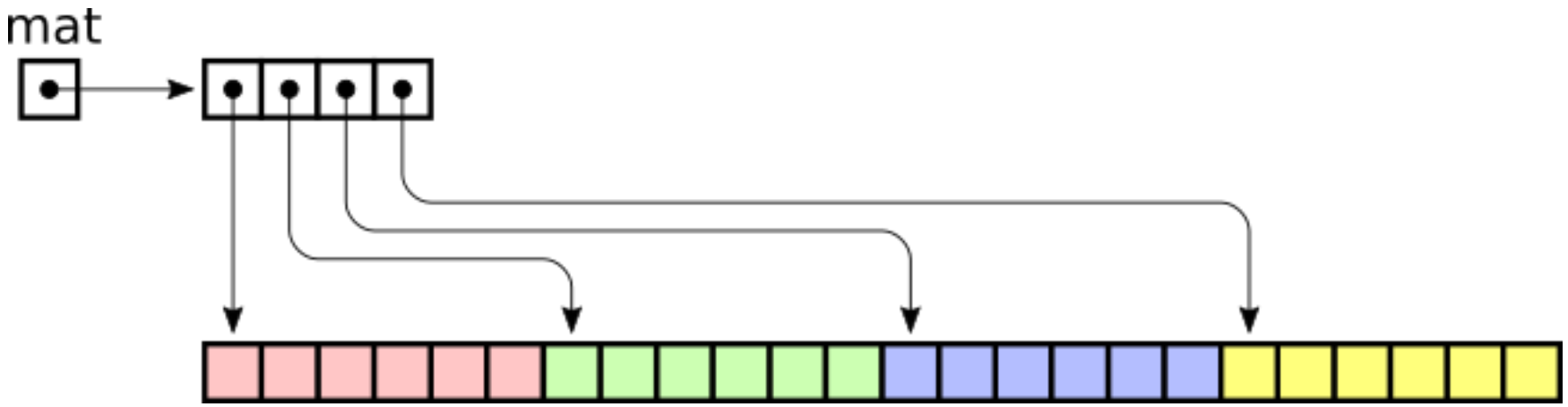
Matriz - Alocação dinâmica I



Matriz - Alocação Dinâmica II



Matriz - Alocação Dinâmica III



Exercício 2

Crie uma estrutura para representar uma pessoa (idade, altura e salário). Na função main, pergunte ao usuário quantas pessoas ele deseja cadastrar. Em seguida, escreva uma função que receba o número de pessoas, aloca um vetor de estruturas, leia os dados de cada pessoa e retorna um ponteiro para esse vetor. Por fim, crie funções que receba esse vetor e calcula as médias de idade, altura e salário.

Boas práticas

https://developer.ibm.com/articles/au-toughgame/